

Arrays Solved Questions

Every problem from Part 13 of the Arrays guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38

PROBLEMS

3

TIERS

14

PATTERNS

129

TESTS PASSING

Prepared for **Hari** · on-campus placement preparation

Companion to **Arrays-Guide.pdf** · code verified against **reference-part13-solutions.cpp**

Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.

Generated 14 September 2026

Contents

Tier 1 — Foundations

- | | |
|----|---|
| 01 | LC 1 · Two Sum |
| 02 | LC 26 · Remove Duplicates from Sorted Array |
| 03 | LC 27 · Remove Element |
| 04 | LC 283 · Move Zeroes |
| 05 | LC 88 · Merge Sorted Array |
| 06 | LC 121 · Best Time to Buy and Sell Stock |
| 07 | LC 53 · Maximum Subarray |
| 08 | LC 217 · Contains Duplicate |
| 09 | LC 169 · Majority Element |
| 10 | LC 268 · Missing Number |
| 11 | LC 136 · Single Number |
| 12 | LC 66 · Plus One |
| 13 | LC 448 · Find All Numbers Disappeared in an Array |

Tier 2 — The placement core

- | | |
|----|---|
| 14 | LC 167 · Two Sum II — Input Array Is Sorted |
| 15 | LC 15 · 3Sum |
| 16 | LC 11 · Container With Most Water |
| 17 | LC 42 · Trapping Rain Water |
| 18 | LC 75 · Sort Colors |
| 19 | LC 209 · Minimum Size Subarray Sum |
| 20 | LC 3 · Longest Substring Without Repeating Characters |
| 21 | LC 424 · Longest Repeating Character Replacement |
| 22 | LC 560 · Subarray Sum Equals K |
| 23 | LC 238 · Product of Array Except Self |
| 24 | LC 152 · Maximum Product Subarray |
| 25 | LC 56 · Merge Intervals |
| 26 | LC 33 · Search in Rotated Sorted Array |
| 27 | LC 153 · Find Minimum in Rotated Sorted Array |
| 28 | LC 34 · Find First and Last Position of Element |

Tier 3 — Harder, but they do appear

- | | |
|----|---------------------------------------|
| 29 | LC 41 · First Missing Positive |
| 30 | LC 287 · Find the Duplicate Number |
| 31 | LC 128 · Longest Consecutive Sequence |

- 32 LC 73 · Set Matrix Zeroes
- 33 LC 48 · Rotate Image
- 34 LC 54 · Spiral Matrix
- 35 LC 189 · Rotate Array
- 36 LC 239 · Sliding Window Maximum
- 37 LC 4 · Median of Two Sorted Arrays
- 38 LC 31 · Next Permutation

Tier 1 — Foundations

Thirteen problems that build the index discipline everything else needs: the write pointer, the hash map, the running-state scan, XOR, and in-place marking. All $O(n)$, nearly all one pass. Do every one of these before moving on.

01 LC 1 · Two Sum

EASY Pattern 12 — Hashing / complement lookup

PROBLEM

Given an array and a `target`, return the indices of the two numbers that add up to `target`. Exactly one answer exists.

APPROACH

Brute force checks all pairs in $O(n^2)$. Instead, walk once and ask *what would complete this element*: for `nums[i]` the partner is `target - nums[i]`. Keep every value seen so far in a hash map from value to index, so the question is answered in $O(1)$.

Complexity $O(n)$ time · $O(n)$ space

C++ SOLUTION

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> seen;           // value -> its index
        for (int i = 0; i < (int)nums.size(); i++) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end()) return {it->second, i};
            seen[nums[i]] = i;                 // insert AFTER the lookup
        }
        return {};
    }
};
```

WATCH OUT

Look up **before** you insert the current element, otherwise an element pairs with itself when `target` is exactly `2 * nums[i]`. Do not sort — sorting destroys the indices you were asked to return.

02 LC 26 · Remove Duplicates from Sorted Array

EASY

Pattern 3 — Two pointers, write index

PROBLEM

The array is sorted. Remove duplicates in place so each value appears once, and return the new length `k`. The first `k` slots must hold the answer.

APPROACH

Write index: `w` is the next output slot, so `v[0..w-1]` is the finished answer. Since the array is sorted, duplicates are adjacent, so a new element is worth keeping exactly when it differs from the last one kept.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int removeDuplicates(vector<int>& v) {
        int n = v.size();
        if (n == 0) return 0;
        int w = 1; // v[0] is always kept
        for (int r = 1; r < n; r++)
            if (v[r] != v[w - 1]) v[w++] = v[r]; // compare with the last KEPT
        return w;
    }
};
```

WATCH OUT

Compare `v[r]` with `v[w-1]` — the last *accepted* value — not with `v[r-1]`. Seed `w = 1` and guard the empty array, since `v[w-1]` would read `v[-1]`.

03 LC 27 · Remove Element

EASY

Pattern 3 — Two pointers, write index

PROBLEM

Remove every occurrence of `val` in place and return the new length. The order of the remaining elements does not matter.

APPROACH

The same write-index skeleton with a different keep-test. Everything that is not `val` is copied forward to `v[w]` and `w` advances; everything else is simply skipped.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int removeElement(vector<int>& v, int val) {
        int w = 0; // next write slot
        for (int r = 0; r < (int)v.size(); r++)
            if (v[r] != val) v[w++] = v[r];
        return w; // w is also the new length
    }
};
```

WATCH OUT

`w` is both the next write slot and the final length — that is not a coincidence, it is the loop invariant `v[0..w-1]` is finished. Do not try to `erase()` inside the loop; that is $O(n)$ per call and invalidates iterators.

04 LC 283 · Move Zeroes

EASY

Pattern 3 — Two pointers, write index

PROBLEM

Move all zeroes to the end while keeping the relative order of the non-zero elements. In place.

APPROACH

Two passes over the same array: compact all non-zeroes to the front with a write index, then fill the tail from `w` to `n-1` with zeroes. A one-pass variant swaps `v[w++]` with `v[r]` instead.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void moveZeroes(vector<int>& v) {
        int n = v.size(), w = 0;
        for (int r = 0; r < n; r++)
            if (v[r] != 0) v[w++] = v[r];           // pass 1: compact the non-zeros
        while (w < n) v[w++] = 0;                  // pass 2: zero-fill the tail
    }
};
```

WATCH OUT

The order of the non-zeroes must be **preserved**, so you cannot swap from the end. The swap version works because `w <= r` always, so you never overwrite an unread element.

05 LC 88 · Merge Sorted Array

EASY

Pattern 2 — Two pointers, filling from the back

PROBLEM

`a` has length `m + n` with its first `m` slots used; `b` has `n` elements. Merge `b` into `a` in place, keeping it sorted.

APPROACH

Merging front to back would overwrite unread elements of `a`. Fill from the **back** instead: compare the two largest remaining values and write the bigger one into `a[w]`, moving `w` leftwards. The free space is always exactly where you are writing.

Complexity $O(m + n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void merge(vector<int>& a, int m, vector<int>& b, int n) {
        int i = m - 1, j = n - 1, w = m + n - 1; // fill from the BACK
        while (j >= 0) {
            if (i >= 0 && a[i] > b[j]) a[w--] = a[i--];
            else a[w--] = b[j--];
        }
    }
};
```

WATCH OUT

Loop `while (j >= 0)`, not until both run out: if `a` empties first the remainder of `b` must still be copied, but if `b` empties first the rest of `a` is already in place. The `i >= 0 &&` guard must come first in the comparison.

06 LC 121 · Best Time to Buy and Sell Stock

EASY Pattern 1 — Linear scan with state

PROBLEM

One buy and one sell, and the buy must come first. Maximise the profit, or return 0.

APPROACH

For each day, the best possible sale today is *today's price minus the cheapest price seen so far*. Carry the running minimum and the running best profit in two variables.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int maxProfit(vector<int>& p) {
        int best = 0, lo = INT_MAX;
        for (int x : p) {
            lo = min(lo, x);           // cheapest day so far
            best = max(best, x - lo);  // sell today against it
        }
        return best;
    }
};
```

WATCH OUT

Update the minimum *before* computing today's profit and the same-day trade contributes 0, which is harmless. Initialise `best = 0`, not `INT_MIN` — a falling market must return 0, not a negative profit. This is Kadane over the day-to-day differences, and saying so is a good interview remark.

07 LC 53 · Maximum Subarray

MEDIUM Pattern 1 — Kadane

PROBLEM

Find the contiguous subarray with the largest sum and return that sum.

APPROACH

Kadane. At each index the best subarray ending here either extends the previous one or starts fresh at this element: `cur = max(v[i], cur + v[i])`. Track the best `cur` ever seen.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int maxSubArray(vector<int>& v) {
        int best = v[0], cur = v[0];           // NOT 0 - all-negative input
        for (int i = 1; i < (int)v.size(); i++) {
            cur = max(v[i], cur + v[i]);        // restart here, or extend
            best = max(best, cur);
        }
        return best;
    }
};
```

WATCH OUT

Initialise `best` and `cur` from `v[0]`, never from 0. With 0 an all-negative array wrongly returns 0; the hidden test is `[-3, -1, -2]`, whose answer is `-1`. To report the indices too, record `start` whenever `cur` restarts.

08 LC 217 · Contains Duplicate

EASY

Pattern 12 — Hashing / membership

PROBLEM

Return true if any value appears at least twice.

APPROACH

Insert each value into an `unordered_set`; `insert` returns a pair whose `.second` is false when the value was already present, so one pass and one lookup per element suffices.

Complexity $O(n)$ time · $O(n)$ space

C++ SOLUTION

```
class Solution {
public:
    bool containsDuplicate(vector<int>& v) {
        unordered_set<int> seen;
        for (int x : v)
            if (!seen.insert(x).second) return true; // insert failed -> already in
        return false;
    }
};
```

WATCH OUT

Sorting and scanning adjacent pairs also works but is $O(n \log n)$ — know both and be able to state the trade: the hash set buys time with memory. Reading `mp[x]` on a map would *insert* a zero; the set's `insert` return value avoids two lookups.

09 LC 169 · Majority Element

EASY

Pattern 1 — Boyer-Moore vote

PROBLEM

One element appears more than $n/2$ times. Find it in $O(1)$ space.

APPROACH

Boyer-Moore voting: hold a candidate and a counter. A matching element increments, a different one decrements, and a counter of zero adopts the next element as the new candidate. The true majority cannot be fully cancelled because it outnumbers everything else combined.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int majorityElement(vector<int>& v) {
        int cand = v[0], count = 0; // Boyer-Moore vote
        for (int x : v) {
            if (count == 0) cand = x;
            count += (x == cand) ? 1 : -1;
        }
        return cand; // a majority is guaranteed; otherwise verify in pass 2
    }
};
```

WATCH OUT

The algorithm is only valid **because a majority is guaranteed**. If the problem does not promise one, add a second pass to count the candidate and verify it exceeds $n/2$. The $n/3$ variant needs two candidates and always needs that verification pass.

10 LC 268 · Missing Number

EASY

Pattern 13 — XOR parity

PROBLEM

The array holds n distinct numbers from the range $0..n$. Return the one that is missing.

APPROACH

XOR every index $0..n-1$ and every value together, seeded with n . Each number that is present appears once as a value and once as an index and cancels itself, so only the missing one survives.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int missingNumber(vector<int>& v) {
        int n = v.size(), res = n;           // seed with n: index n has no v
        for (int i = 0; i < n; i++) res ^= i ^ v[i];
        return res;                          // every present pair cancels
    }
};
```

WATCH OUT

The sum formula $n*(n+1)/2 - \text{sum}$ is easier to explain but **overflows** for large n ; XOR never can. Offering both, and naming the overflow difference, is exactly the comparison interviewers are listening for.

11 LC 136 · Single Number

EASY

Pattern 13 — XOR parity

PROBLEM

Every element appears twice except one. Find it, in $O(1)$ space.

APPROACH

XOR the whole array. Because $x \oplus x == 0$ and $x \oplus 0 == x$, and XOR is commutative and associative, every pair annihilates regardless of position and the loner is what remains.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int singleNumber(vector<int>& v) {
        int res = 0;
        for (int x : v) res ^= x;           //  $x \oplus x = 0$ ,  $x \oplus 0 = x$ 
        return res;
    }
};
```

WATCH OUT

A hash map works but costs $O(n)$ space, which the problem forbids. For the variant where two numbers appear once (LC 260), XOR everything to get $a \oplus b$, isolate the lowest set bit with $x \& -x$, and split the array on that bit.

12 LC 66 • Plus One

EASY Pattern 1 — Linear scan with carry

PROBLEM

A non-negative integer is stored as an array of digits, most significant first. Add one and return the digit array.

APPROACH

Walk from the last digit backwards. Anything below 9 can absorb the increment and you return immediately; a 9 becomes 0 and the carry continues left. If the loop finishes, every digit was a 9, so prepend a 1.

Complexity $O(n)$ time · $O(1)$ extra space ($O(n)$ only in the all-nines case)

C++ SOLUTION

```
class Solution {
public:
    vector<int> plusOne(vector<int>& d) {
        for (int i = (int)d.size() - 1; i >= 0; i--) {
            if (d[i] < 9) { d[i]++; return d; } // no carry -> done
            d[i] = 0; // 9 -> 0, carry continues
        }
        d.insert(d.begin(), 1); // all nines: 999 -> 1000
        return d;
    }
};
```

WATCH OUT

The all-nines case is the whole problem: `999` must become `1000`, which is *longer* than the input. Do not convert to `int` or `long long` — the array can hold hundreds of digits, which is precisely why the problem is phrased this way.

13 LC 448 · Find All Numbers Disappeared in an Array

EASY Pattern 13 — In-place marking

PROBLEM

An array of length `n` holds values in `1..n`, some appearing twice. Return every value in `1..n` that does not appear, in $O(1)$ extra space.

APPROACH

Use the sign bit as a 'seen' flag. For each element, negate the value sitting at index `|v[i]| - 1`. After one pass every slot still holding a positive number corresponds to a value that was never visited.

Complexity $O(n)$ time · $O(1)$ extra space (the output does not count)

C++ SOLUTION

```
class Solution {
public:
    vector<int> findDisappearedNumbers(vector<int>& v) {
        int n = v.size();
        for (int i = 0; i < n; i++) {
            int idx = abs(v[i]) - 1;           // abs: v[i] may be marked
            if (v[idx] > 0) v[idx] = -v[idx];   // mark "value idx+1 is present"
        }
        vector<int> res;
        for (int i = 0; i < n; i++) if (v[i] > 0) res.push_back(i + 1);
        return res;
    }
};
```

WATCH OUT

`abs(v[i])` is mandatory — by the time you reach index `i`, `v[i]` may itself already have been flipped negative, and a negative index is undefined behaviour. Swapping the final test to `v[i] < 0` solves LC 442 (find all duplicates) with the same code.

Tier 2 — The placement core

Fifteen problems, each the home ground of a named pattern: opposite-ends two pointers, the Dutch flag, both flavours of sliding window, prefix sums with a hash map, the prefix/suffix double sweep, interval sweeping and the three binary-search shapes. This is the tier that decides your placement result.

14 LC 167 · Two Sum II — Input Array Is Sorted

MEDIUM Pattern 2 — Two pointers, opposite ends

PROBLEM

The array is sorted ascending. Return the **1-indexed** positions of the two values summing to `target`.

APPROACH

Start at both ends. If the sum is too small only the left pointer can help, because `v[r]` is already the largest available partner for `v[l]`; if it is too large, only the right pointer can. Each step discards exactly one candidate and never discards a solution.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    vector<int> twoSum(vector<int>& v, int target) {
        int l = 0, r = (int)v.size() - 1;
        while (l < r) {
            int s = v[l] + v[r];
            if (s == target) return {l + 1, r + 1}; // this problem is 1-indexed
            if (s < target) l++; // need more -> raise the low end
            else r--; // need less -> lower the high end
        }
        return {};
    }
};
```

WATCH OUT

This problem is **1-indexed** — return `{l+1, r+1}`. Use `while (l < r)`, not `<=`, since the two elements must be distinct. Being able to state *why* discarding an end is safe is the actual interview question.

15 LC 15 · 3Sum

MEDIUM Pattern 2 — Sort + two pointers

PROBLEM

Return all unique triplets summing to zero. No duplicate triplets in the output.

APPROACH

Sort, then fix the first element `v[i]` and run the opposite-ends two-pointer scan over the rest looking for `-v[i]`. Sorting is what makes both the two-pointer move and the duplicate-skipping possible.

Complexity $O(n^2)$ time · $O(1)$ extra space beyond the output

C++ SOLUTION

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& v) {
        sort(v.begin(), v.end());
        int n = v.size();
        vector<vector<int>> res;
        for (int i = 0; i + 2 < n; i++) {
            if (v[i] > 0) break; // sorted: no triple can reach 0
            if (i > 0 && v[i] == v[i - 1]) continue; // skip duplicate FIRST element
            int l = i + 1, r = n - 1;
            while (l < r) {
                long long s = (long long)v[i] + v[l] + v[r];
                if (s == 0) {
                    res.push_back({v[i], v[l], v[r]});
                    l++; r--;
                    while (l < r && v[l] == v[l - 1]) l++; // skip duplicate SECOND
                    while (l < r && v[r] == v[r + 1]) r--; // skip duplicate THIRD
                } else if (s < 0) l++;
                else r--;
            }
        }
        return res;
    }
};
```

WATCH OUT

There are **three** de-duplication points, and each compares against the element just left behind: `v[i] == v[i-1]` for the first, and `v[l] == v[l-1]` / `v[r] == v[r+1]` after recording a hit. Sum into a `long long`; three `int`s near the limits can overflow. `if (v[i] > 0) break;` is a free early exit on sorted input.

16 LC 11 · Container With Most Water

MEDIUM

Pattern 2 — Two pointers, greedy move

PROBLEM

Each element is a vertical wall height. Choose two walls so the water they hold — $\text{min}(\text{height}) * \text{distance}$ — is maximised.

APPROACH

Start at the widest pair and always move the **shorter** wall inward. Moving the taller one cannot help: the width strictly shrinks while the height stays capped by the shorter wall, so no better area can appear.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int maxArea(vector<int>& h) {
        int l = 0, r = (int)h.size() - 1, best = 0;
        while (l < r) {
            best = max(best, min(h[l], h[r]) * (r - l));
            if (h[l] < h[r]) l++;           // move the SHORTER wall
            else r--;                       // the taller one caps nothing
        }
        return best;
    }
};
```

WATCH OUT

The greedy exchange argument above is the whole answer — an interviewer will ask you to justify discarding the shorter wall. When the two heights are equal, either move is safe. Note that this problem ignores the walls in between, unlike Trapping Rain Water.

17 LC 42 · Trapping Rain Water

HARD

Pattern 2 — Two pointers + running max

PROBLEM

Given an elevation map, compute how much rain water is trapped between the bars.

APPROACH

Water above bar `i` is `min(maxLeft, maxRight) - h[i]`. Two pointers let you compute that without precomputing both arrays: whichever side is currently shorter is the limiting one, so its running max is already final and you can settle that column and step inward.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int trap(vector<int>& h) {
        int n = h.size();
        if (n == 0) return 0;
        int l = 0, r = n - 1, lmax = 0, rmax = 0, water = 0;
        while (l < r) {
            if (h[l] < h[r]) {                // the left side is the limit
                lmax = max(lmax, h[l]);
                water += lmax - h[l];
                l++;
            } else {
                rmax = max(rmax, h[r]);
                water += rmax - h[r];
                r--;
            }
        }
        return water;
    }
};
```

WATCH OUT

Add `lmax - h[l]` after updating `lmax`, so the contribution is never negative. The prefix/suffix-array version ($O(n)$ space) and the monotonic-stack version are both acceptable — know at least two and say which you are writing and why.

18 LC 75 · Sort Colors

MEDIUM Pattern 4 — Dutch national flag

PROBLEM

Sort an array of 0s, 1s and 2s in one pass, in place, without a library sort.

APPROACH

Three pointers maintaining four regions: `[0, low)` all 0s, `[low, mid)` all 1s, `[mid, high]` unknown, `(high, n)` all 2s. A 0 swaps to `low` and both advance, a 1 just advances `mid`, a 2 swaps to `high` and `high` shrinks.

Complexity $O(n)$ time · one pass, $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void sortColors(vector<int>& v) {
        int low = 0, mid = 0, high = (int)v.size() - 1;
        // [0, low) = 0s   [low, mid) = 1s   [mid, high] = unknown   (high, n) = 2s
        while (mid <= high) {
            if (v[mid] == 0) swap(v[low++], v[mid++]);
            else if (v[mid] == 1) mid++;
            else swap(v[mid], v[high--]); // mid does NOT advance
        }
    }
};
```

WATCH OUT

`mid` must not advance after swapping with `high` — the value that came back is from the unexamined region and has to be inspected next iteration. After swapping with `low` the incoming value is a known 1, so advancing is safe. Loop `while (mid <= high)`, inclusive, since `high` is still unknown.

19 LC 209 · Minimum Size Subarray Sum

MEDIUM Pattern 6 — Variable window, shortest

PROBLEM

All values are positive. Find the length of the shortest contiguous subarray whose sum is at least `target`, or 0 if none exists.

APPROACH

Variable window, minimising flavour: grow on the right, and **while the window is valid**, record the length and then shrink from the left to hunt for something shorter.

Complexity $O(n)$ time · $O(1)$ space — each index enters and leaves the window once

C++ SOLUTION

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& v) {
        int n = v.size(), l = 0, best = INT_MAX;
        long long sum = 0;
        for (int r = 0; r < n; r++) {
            sum += v[r];
            while (sum >= target) {                // valid -> record, then shrink
                best = min(best, r - l + 1);
                sum -= v[l++];
            }
        }
        return best == INT_MAX ? 0 : best;
    }
};
```

WATCH OUT

For a *shortest* answer the recording happens **inside** the shrink loop; for a *longest* answer it happens after it. Return 0, not `INT_MAX`, when nothing qualifies. The window is only valid here because the values are positive — with negatives this needs prefix sums.

20 LC 3 · Longest Substring Without Repeating Characters

MEDIUM Pattern 6 — Variable window, longest

PROBLEM

Find the length of the longest substring containing no repeated character.

APPROACH

Variable window with a 128-entry frequency table. After adding `s[r]`, the only character that can possibly have a count above 1 is `s[r]` itself, so shrink from the left until its count drops back to 1, then record `r - l + 1`.

Complexity $O(n)$ time · $O(1)$ space (the table is a fixed 128 entries)

C++ SOLUTION

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int cnt[128] = {0};
        int l = 0, best = 0;
        for (int r = 0; r < (int)s.size(); r++) {
            cnt[(unsigned char)s[r]]++;
            while (cnt[(unsigned char)s[r]] > 1)    // only the new char can clash
                cnt[(unsigned char)s[l++]]--;
            best = max(best, r - l + 1);
        }
        return best;
    }
};
```

WATCH OUT

Testing only `cnt[s[r]] > 1` rather than rescanning the table is what keeps this $O(n)$. Cast to `unsigned char` before indexing: plain `char` is signed, so a byte above 127 would index negatively.

21 LC 424 · Longest Repeating Character Replacement

MEDIUM Pattern 6 — Variable window + frequency

PROBLEM

You may change at most k characters. Find the longest substring that can be made of a single repeated letter.

APPROACH

A window is valid when $(\text{length}) - (\text{count of its most frequent letter}) \leq k$, since that difference is exactly how many characters you would have to replace. Grow on the right, shrink while invalid.

Complexity $O(n)$ time · $O(1)$ space (26 counters)

C++ SOLUTION

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        int cnt[26] = {0};
        int l = 0, maxCount = 0, best = 0;
        for (int r = 0; r < (int)s.size(); r++) {
            maxCount = max(maxCount, ++cnt[s[r] - 'A']);
            // window is valid while (length - most frequent) <= k
            while ((r - l + 1) - maxCount > k) cnt[s[l++] - 'A']--;
            best = max(best, r - l + 1);
        }
        return best;
    }
};
```

WATCH OUT

`maxCount` is kept as a running maximum and never decreased. That looks wrong but is not: a stale, too-large `maxCount` can only keep the window from shrinking, and a window shorter than the best already found can never improve the answer. If that argument feels shaky, recompute the max over 26 entries — still fast, and obviously correct.

22 LC 560 · Subarray Sum Equals K

MEDIUM Pattern 8 — Prefix sum + hash map

PROBLEM

Count the contiguous subarrays whose sum is exactly `k`. Values may be **negative**.

APPROACH

$\text{sum}(i..j) == k$ is the same as $P[j+1] - P[i] == k$, i.e. $P[i] == P[j+1] - k$. Scan once keeping a map from each prefix value to how many times it has occurred; at each step add the count of `pre - k`.

Complexity $O(n)$ time · $O(n)$ space

C++ SOLUTION

```
class Solution {
public:
    int subarraySum(vector<int>& v, int k) {
        unordered_map<long long, int> seen;
        seen[0] = 1; // the empty prefix – essential
        long long pre = 0;
        int count = 0;
        for (int x : v) {
            pre += x;
            auto it = seen.find(pre - k); // look up BEFORE inserting
            if (it != seen.end()) count += it->second;
            seen[pre]++;
        }
        return count;
    }
};
```

WATCH OUT

Seed the map with `seen[0] = 1` — the empty prefix — or every subarray starting at index 0 is missed; test with `v = [3], k = 3`. Look up before inserting. A sliding window is *wrong* here because negatives break the monotonicity that justifies shrinking.

23 LC 238 · Product of Array Except Self

MEDIUM Pattern 7 — Prefix/suffix sweep

PROBLEM

Return an array where `res[i]` is the product of every element except `v[i]`. No division, $O(1)$ extra space beyond the output.

APPROACH

Two sweeps. The forward pass leaves 'product of everything to my left' in `res`. The backward pass multiplies in 'product of everything to my right', carried in a single running variable.

Complexity $O(n)$ time · $O(1)$ extra space (the output array is not counted)

C++ SOLUTION

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& v) {
        int n = v.size();
        vector<int> res(n, 1);
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * v[i - 1]; // left products
        int suf = 1;
        for (int i = n - 1; i >= 0; i--) { // right sweep
            res[i] *= suf;
            suf *= v[i];
        }
        return res;
    }
};
```

WATCH OUT

Division is banned precisely because of zeros — one zero makes the quotient undefined and two zeros make every answer zero. The left-then-right double sweep is a pattern in its own right; the same shape solves Trapping Rain Water and Candy.

24 LC 152 · Maximum Product Subarray

MEDIUM

Pattern 1 — Kadane with min and max

PROBLEM

Find the contiguous subarray with the largest **product**.

APPROACH

Kadane's structure, but a large negative times a negative becomes a large positive — so carry both the running maximum and the running minimum ending here, and swap their roles whenever the current element is negative.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int maxProduct(vector<int>& v) {
        int best = v[0], hi = v[0], lo = v[0];
        for (int i = 1; i < (int)v.size(); i++) {
            if (v[i] < 0) swap(hi, lo);           // a negative flips the roles
            hi = max(v[i], hi * v[i]);
            lo = min(v[i], lo * v[i]);
            best = max(best, hi);
        }
        return best;
    }
};
```

WATCH OUT

Swap **before** the two updates, not after. Zeros need no special case: both running values collapse to 0 and the next element restarts the run. Initialise all three variables from `v[0]`, as in LC 53.

25 LC 56 · Merge Intervals

MEDIUM Pattern 9 — Interval sweep

PROBLEM

Merge all overlapping intervals and return the non-overlapping set covering the same span.

APPROACH

Sort by start. Sweep left to right: if the current interval starts at or before the end of the last kept one, extend that one; otherwise push the current interval as a new block.

Complexity $O(n \log n)$ time (dominated by the sort), $O(1)$ extra space beyond the output

C++ SOLUTION

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& iv) {
        sort(iv.begin(), iv.end());           // by start, then by end
        vector<vector<int>> res;
        for (auto& cur : iv) {
            if (!res.empty() && cur[0] <= res.back()[1])
                res.back()[1] = max(res.back()[1], cur[1]); // max: cur may be inside
            else
                res.push_back(cur);
        }
        return res;
    }
};
```

WATCH OUT

Extend with `max(res.back()[1], cur[1])`, never plain assignment — `[1,10]` followed by `[2,3]` must stay `[1,10]`. Use `<=` so touching intervals like `[1,4]` and `[4,5]` merge, unless the statement says otherwise. For 'keep the most non-overlapping intervals', sort by **end** instead.

26 LC 33 · Search in Rotated Sorted Array

MEDIUM Pattern 10 — Binary search, rotated

PROBLEM

A sorted array of distinct values was rotated at an unknown pivot. Find `target` in $O(\log n)$, or return `-1`.

APPROACH

At every split at least one half is still properly sorted. Detect which by comparing `v[lo]` with `v[mid]`, then test whether the target lies inside that sorted half's range: if it does, search there; otherwise search the other half.

Complexity $O(\log n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int search(vector<int>& v, int target) {
        int lo = 0, hi = (int)v.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (v[mid] == target) return mid;
            if (v[lo] <= v[mid]) { // left half sorted
                if (v[lo] <= target && target < v[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else { // right half sorted
                if (v[mid] < target && target <= v[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }
};
```

WATCH OUT

`v[lo] <= v[mid]` needs the `<=`: when only two elements remain, `mid == lo` and the one-element left half is trivially sorted. With duplicates (LC 81) the test can be uninformative — `[3,3,3,1,3]` — and the fix, `hi--`, degrades the worst case to $O(n)$. Say that out loud when asked.

27 LC 153 · Find Minimum in Rotated Sorted Array

MEDIUM Pattern 10 — Binary search, rotated

PROBLEM

Return the smallest element of a rotated sorted array of distinct values, in $O(\log n)$.

APPROACH

There is no target, so compare against the right end. If `v[mid] > v[hi]` the rotation point — and therefore the minimum — is strictly to the right; otherwise `mid` is itself a candidate and the search continues left with `hi = mid`.

Complexity $O(\log n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int findMin(vector<int>& v) {
        int lo = 0, hi = (int)v.size() - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (v[mid] > v[hi]) lo = mid + 1;    // minimum is strictly right
            else hi = mid;                      // mid could BE the minimum
        }
        return v[lo];                          // compare with hi, never with lo
    }
};
```

WATCH OUT

Compare with `v[hi]`, never with `v[lo]`. Comparing against the left end fails on an unrotated array like `[1,2,3]`. This is the `while (lo < hi)` template, so `hi = mid` (not `mid-1`) and the answer is `v[lo]` after the loop.

28 LC 34 · Find First and Last Position of Element

MEDIUM Pattern 10 — Binary search, boundaries

PROBLEM

Return the first and last index of `target` in a sorted array, or `{-1, -1}`.

APPROACH

Two boundary searches with one helper. `lower(x)` is the first index with `v[i] >= x`; the first occurrence is `lower(target)` and the last is `lower(target + 1) - 1`, since the first index greater than `target` is one past the final occurrence.

Complexity $O(\log n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    vector<int> searchRange(vector<int>& v, int target) {
        int lo = lower(v, target);           // first index >= target
        if (lo == (int)v.size() || v[lo] != target) return {-1, -1};
        return {lo, lower(v, target + 1) - 1}; // first > target, minus one
    }
private:
    int lower(vector<int>& v, int x) {         // binary search template 2
        int lo = 0, hi = v.size();           // hi = n: "not found" is valid
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (v[mid] >= x) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
};
```

WATCH OUT

The helper's `hi` starts at `n`, not `n-1`, because 'not present, belongs at the end' is a legitimate answer. Check both `lo == n` and `v[lo] != target` before trusting the result. `lower(target+1)` is `upper_bound` without writing a second function.

Tier 3 — Harder, but they do appear

Ten problems where the naive answer is easy and the required one is not: $O(1)$ space when you wanted a hash set, read-only when you wanted to sort, one pass when you wanted two. Cyclic sort, Floyd on an array, in-place matrix markers, the monotonic deque, and binary search on a partition.

29 LC 41 · First Missing Positive

HARD Pattern 13 — Cyclic sort

PROBLEM

Find the smallest missing **positive** integer, in $O(n)$ time and $O(1)$ space

APPROACH

The answer must lie in $[1..n+1]$, so only values in $[1..n]$ matter and each has a natural home: value x belongs at index $x-1$. Keep swapping elements into their homes, then scan for the first index whose value is wrong — that index plus one is the answer.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    int firstMissingPositive(vector<int>& v) {
        int n = v.size();
        for (int i = 0; i < n; i++)
            // send v[i] to its home slot v[i]-1 until the one at i belongs there
            while (v[i] > 0 && v[i] <= n && v[v[i] - 1] != v[i])
                swap(v[i], v[v[i] - 1]); // compare VALUES, not v[i] != i+1
        for (int i = 0; i < n; i++)
            if (v[i] != i + 1) return i + 1; // first slot holding the wrong value
        return n + 1;
    }
};
```

WATCH OUT

The nested `while` is **not** $O(n^2)$: every successful swap places one value permanently, so there are at most n swaps overall — have that argument ready, it is always the follow-up. The loop test must be `v[v[i]-1] != v[i]`, comparing values; writing `v[i] != i+1` instead loops forever on duplicates.

30 LC 287 · Find the Duplicate Number

MEDIUM Pattern 13 — Floyd cycle on an array

PROBLEM

An array of $n+1$ values in $1..n$ has exactly one repeated value. Find it without modifying the array and in $O(1)$ space.

APPROACH

Read $v[i]$ as 'the next index' and the array becomes a linked list. Two arrows landing on the same node means a cycle, and its entrance is the duplicate. Floyd: advance slow by one and fast by two until they meet, then restart one at index 0 and step both by one — they meet at the entrance.

Complexity $O(n)$ time · $O(1)$ space, input untouched

C++ SOLUTION

```
class Solution {
public:
    int findDuplicate(vector<int>& v) {
        int slow = v[0], fast = v[0];
        do {
            slow = v[slow];
            fast = v[v[fast]];
        } while (slow != fast);
        slow = v[0];
        while (slow != fast) {
            slow = v[slow];
            fast = v[fast];
        }
        return slow;
    }
};
```

WATCH OUT

The two constraints together — read-only and $O(1)$ space — are what rule out sorting, a hash set, and negation marking. Phase 2 must restart from $v[0]$ and move **both** pointers one step at a time; that is the part people misremember.

31 LC 128 · Longest Consecutive Sequence

MEDIUM Pattern 12 — Hashing / set walk

PROBLEM

Find the length of the longest run of consecutive integers in an unsorted array. $O(n)$ required.

APPROACH

Put everything in a hash set. For each value, only start counting if `x-1` is absent — that makes `x` the start of its run — then walk `x+1, x+2, ...` upward while the set contains them.

Complexity $O(n)$ time · $O(n)$ space

C++ SOLUTION

```
class Solution {
public:
    int longestConsecutive(vector<int>& v) {
        unordered_set<int> s(v.begin(), v.end());
        int best = 0;
        for (int x : s) {
            if (s.count(x - 1)) continue; // only walk from a sequence START
            int len = 1;
            while (s.count(x + len)) len++; // keeps the whole loop O(n)
            best = max(best, len);
        }
        return best;
    }
};
```

WATCH OUT

The `if (s.count(x-1)) continue;` guard is what keeps this linear: the inner walk only ever runs from a run's smallest element, so across the whole loop each element is visited at most once. Without it the same run is re-walked from every member and you are back to $O(n^2)$. Sorting solves it too, but at $O(n \log n)$, which the problem forbids.

32 LC 73 · Set Matrix Zeroes

MEDIUM Pattern 14 — Matrix, in-place markers

PROBLEM

If a cell is 0, set its entire row and column to 0 — in place, using $O(1)$ extra space.

APPROACH

Zeroing as you go is wrong because written zeroes are indistinguishable from original ones. Use the first row and first column as the marker buffers, plus one boolean for the first column itself (since `g[0][0]` would otherwise have to record two different facts).

Complexity $O(m \cdot n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void setZeroes(vector<vector<int>>& g) {
        int m = g.size(), n = g[0].size();
        bool firstCol = false;
        for (int i = 0; i < m; i++) {           // pass 1: record into row 0 / col 0
            if (g[i][0] == 0) firstCol = true;
            for (int j = 1; j < n; j++)
                if (g[i][j] == 0) { g[i][0] = 0; g[0][j] = 0; }
        }
        for (int i = m - 1; i >= 0; i--) {      // pass 2: BACKWARDS, markers last
            for (int j = n - 1; j >= 1; j--)
                if (g[i][0] == 0 || g[0][j] == 0) g[i][j] = 0;
            if (firstCol) g[i][0] = 0;
        }
    }
};
```

WATCH OUT

The second pass must run **backwards**, bottom-right to top-left, so the markers are consumed last. Going forwards destroys the marker row before you have finished reading it. The $O(m+n)$ version with two `vector<bool>`s is much easier — write that first, then optimise if asked.

33 LC 48 · Rotate Image

MEDIUM Pattern 14 — Matrix, transpose + reverse

PROBLEM

Rotate an $n \times n$ matrix by 90° clockwise, in place.

APPROACH

Transpose (reflect across the main diagonal), then reverse each row. Transposing turns rows into columns and the row reversal puts them in the right order, which together is exactly a clockwise quarter turn.

Complexity $O(n^2)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void rotate(vector<vector<int>>& g) {
        int n = g.size();
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++) // j = i+1: upper triangle only
                swap(g[i][j], g[j][i]); // transpose
        for (auto& row : g) reverse(row.begin(), row.end());
    }
};
```

WATCH OUT

j must start at i + 1. Starting at 0 swaps every pair twice and leaves the matrix unchanged — the classic silent failure here. For anticlockwise, transpose and then reverse the *order of the rows* instead.

34 LC 54 · Spiral Matrix

MEDIUM Pattern 14 — Matrix, shrinking boundaries

PROBLEM

Return all elements of an $m \times n$ matrix in spiral order.

APPROACH

Hold four boundaries — `top`, `bot`, `left`, `right` — and peel one ring per iteration: left-to-right along the top, top-to-bottom down the right, right-to-left along the bottom, bottom-to-top up the left, shrinking the relevant boundary after each leg.

Complexity $O(m \cdot n)$ time · $O(1)$ extra space beyond the output

C++ SOLUTION

```
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& g) {
        vector<int> res;
        if (g.empty() || g[0].empty()) return res;
        int top = 0, bot = g.size() - 1, left = 0, right = g[0].size() - 1;
        while (top <= bot && left <= right) {
            for (int j = left; j <= right; j++) res.push_back(g[top][j]);
            top++;
            for (int i = top; i <= bot; i++) res.push_back(g[i][right]);
            right--;
            if (top <= bot) { // guard: one row left
                for (int j = right; j >= left; j--) res.push_back(g[bot][j]);
                bot--;
            }
            if (left <= right) { // guard: one column left
                for (int i = bot; i >= top; i--) res.push_back(g[i][left]);
                left++;
            }
        }
        return res;
    }
};
```

WATCH OUT

The two `if` guards before the bottom and left legs are the whole difficulty. Without them a single remaining row or column is traversed twice and you emit duplicates. Test on `[[1,2,3]]` and `[[1],[2],[3]]` — those two inputs catch every buggy version of this.

35 LC 189 · Rotate Array

MEDIUM

Pattern 3 — In-place rearrangement

PROBLEM

Rotate the array right by k steps, in place with $O(1)$ extra space.

APPROACH

Reverse the whole array, then reverse the first k elements, then reverse the remaining $n - k$. The full reversal puts the last k elements at the front but backwards, and the two partial reversals repair the order of each block.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void rotate(vector<int>& v, int k) {
        int n = v.size();
        if (n == 0) return;
        k %= n;
        reverse(v.begin(), v.end());
        reverse(v.begin(), v.begin() + k);
        reverse(v.begin() + k, v.end());
    }
};
```

WATCH OUT

$k \% n$ first — k is routinely larger than n in the tests, and $v.begin() + k$ past the end is undefined behaviour. Guard $n == 0$ before the modulo, since $\% 0$ crashes. The same triple reversal rotates a string.

36 LC 239 · Sliding Window Maximum

HARD Pattern 6 — Monotonic deque window

PROBLEM

Return the maximum of every window of size `k` as it slides across the array.

APPROACH

Keep a deque of **indices** whose values are strictly decreasing. Before pushing `i`, pop from the back every index whose value is no greater than `v[i]` — those can never be a maximum again while `i` is in the window. Pop from the front when an index falls out of range; the front is then always the answer.

Complexity $O(n)$ time · $O(k)$ space — each index is pushed once and popped once

C++ SOLUTION

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& v, int k) {
        deque<int> dq; // INDICES, values decreasing
        vector<int> res;
        for (int i = 0; i < (int)v.size(); i++) {
            if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // expired
            while (!dq.empty() && v[dq.back()] <= v[i]) dq.pop_back();
            dq.push_back(i);
            if (i >= k - 1) res.push_back(v[dq.front()]);
        }
        return res;
    }
};
```

WATCH OUT

Store **indices, not values**, or you cannot tell when an entry has expired. Do the expiry pop (`dq.front() <= i - k`) before reading the answer, and only start recording once `i >= k - 1`.

37 LC 4 · Median of Two Sorted Arrays

HARD Pattern 11 — Binary search on a partition

PROBLEM

Find the median of two sorted arrays in $O(\log(m+n))$.

APPROACH

Do not merge. Binary search *how many* elements of the combined left half come from the shorter array; that count fixes the number taken from the other. The split is correct when $L1 \leq R2$ && $L2 \leq R1$, and the median then comes from the four boundary values.

Complexity $O(\log(\min(m, n)))$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& a, vector<int>& b) {
        if (a.size() > b.size()) return findMedianSortedArrays(b, a); // search shorter
        int m = a.size(), n = b.size(), half = (m + n + 1) / 2;
        int lo = 0, hi = m;
        while (lo <= hi) {
            int i = lo + (hi - lo) / 2; // take i from a, j from b
            int j = half - i;
            int L1 = (i == 0) ? INT_MIN : a[i - 1];
            int R1 = (i == m) ? INT_MAX : a[i];
            int L2 = (j == 0) ? INT_MIN : b[j - 1];
            int R2 = (j == n) ? INT_MAX : b[j];
            if (L1 > R2) hi = i - 1; // took too many from a
            else if (L2 > R1) lo = i + 1; // took too few from a
            else {
                if ((m + n) % 2) return L1 > L2 ? L1 : L2;
                int left = L1 > L2 ? L1 : L2;
                int right = R1 < R2 ? R1 : R2;
                return (left + right) / 2.0;
            }
        }
        return 0.0;
    }
};
```

WATCH OUT

Always binary search the **shorter** array, or the derived index can fall outside the other array. Use `INT_MIN` / `INT_MAX` as sentinels when a partition is empty at either end — that is what removes every special case. `half = (m+n+1)/2` with the `+1` makes the odd-length case fall out as `max(L1, L2)`.

38 LC 31 · Next Permutation

MEDIUM Pattern 3 — In-place index scan

PROBLEM

Rearrange the array into the next lexicographically greater permutation, in place. If none exists, sort it ascending.

APPROACH

Scan from the right for the first index `i` with `v[i] < v[i+1]` — the pivot. Everything after it is non-increasing and therefore already maximal. Swap the pivot with the rightmost value greater than it, then reverse the tail, which turns the largest arrangement of that suffix into the smallest.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    void nextPermutation(vector<int>& v) {
        int n = v.size(), i = n - 2;
        while (i >= 0 && v[i] >= v[i + 1]) i--;           // 1. rightmost rise v[i]<v[i+1]
        if (i >= 0) {
            int j = n - 1;
            while (v[j] <= v[i]) j--;                     // 2. rightmost value > v[i]
            swap(v[i], v[j]);
        }
        reverse(v.begin() + i + 1, v.end());              // 3. the tail is descending
    }
};
```

WATCH OUT

The tail reversal is what makes the result the *next* permutation rather than merely a greater one. When no pivot exists (`i < 0`) the whole array is descending, and the unconditional final reverse already produces the ascending answer — so it must sit outside the `if`.